Natural Sorting

We saw earlier that SORT_STRING caused elements to be arranged in alphabetical order. If those strings contain only alphabet letters, the results are generally intuitive. However, if strings contain digits, then the sorting sometimes looks a little off.

Imagine a driver traveling at 70 mph needing to read information quickly from some smart sign on the roadside. Let's say she's looking for a particular numbered road and only has a few seconds to do so. Take a look at the following:

```
$motorways = ['M1', 'M11', 'M6', 'M20', 'M4', 'M23'];

sort($motorways, SORT_STRING);
print getMotorways($motorways) . "\n";

natsort($motorways);
print getMotorways($motorways) . "\n";

function getMotorways (array $motorways): string {
    $str = '';

    foreach ($motorways as $m) {
        $str .= "$m, ";
    }

    return rtrim($str, ', ');
};

Output:
M1, M11, M20, M23, M4, M6
M1, M4, M6, M11, M20, M23
```

$motorways is an array of strings corresponding to some of the finest roads in the United Kingdom. Let's pretend that this data arrives into our system in some random order and our code is used to generate the list of roads on our hypothetical smart sign. Here we're comparing the results of sort($array, SORT_STRING), and natsort($array): the results are quite different.

The reason the two functions produce these results is because an alphabetical (SORT_STRING) comparison treats each digit of a number individually rather than atomically, whereas a natural sort treats consecutive digits as numbers. For example, 'A100' is